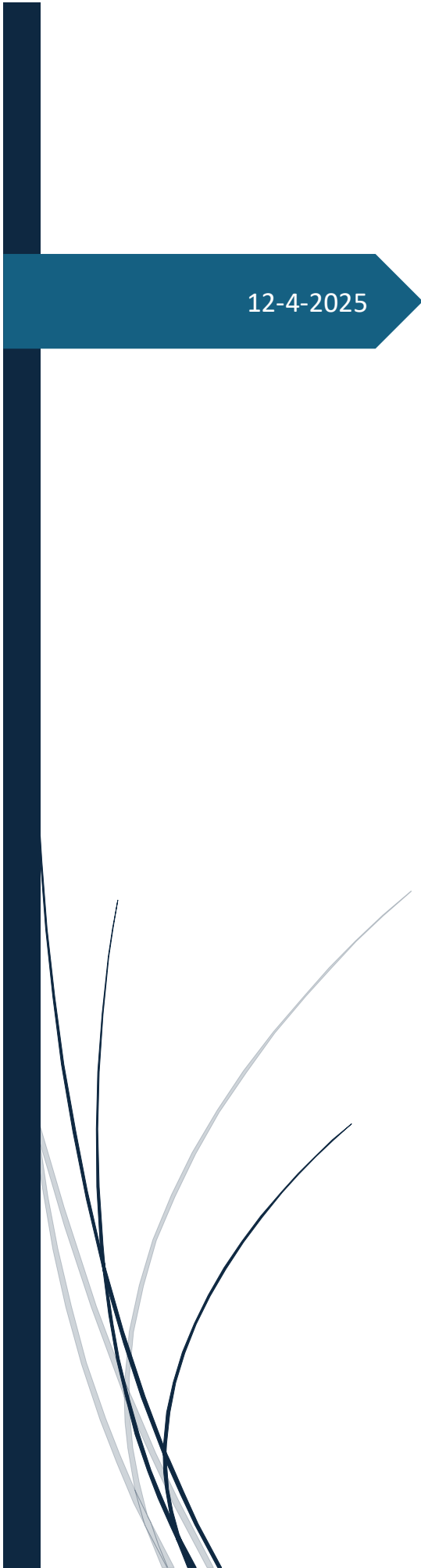




12-4-2025

Reporte: Monitoreo de sistema personalizado

Sistemas Avanzados

PADILLA MIRANDA, JULIO
CESAR

217522132

Introducción

Con el objetivo de poner en práctica lo aprendido de rust se desarrollará un *Visualizador de Métricas de Sistema Avanzado*. Este proyecto se divide en dos partes: en primer lugar, un programa escrito en Rust que se ejecuta al inicio del sistema para capturar en tiempo real métricas detalladas como uso de CPU por núcleo, frecuencia, memoria, tráfico de red, actividad de disco y procesos más demandantes. Estos datos se almacenan de forma histórica, cada 5 a 10 minutos, durante un periodo de hasta cuatro días. Posteriormente, utilizando Python, se procesan estos datos para generar visualizaciones, estadísticas y correlaciones.

Objetivo

Crear un programa que monitoree y analice métricas de sistema en tiempo real (más allá del TaskManager), almacene datos históricos (2 días), y genere estadísticas y proyecciones.

Desarrollo

Para el guardar los datos hice uso del sqlite, con una sola tabla para guardar todas las metricas.

```
// Creacion de la tabla
conn.execute(
    sql: "CREATE TABLE IF NOT EXISTS metrics (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        timestamp TEXT NOT NULL,
        cpu_uso TEXT NOT NULL,
        cpu_frecuencia TEXT NOT NULL,
        mem_fisica REAL,
        mem_swap REAL,
        interface_nombre TEXT,
        data_down TEXT NOT NULL,
        data_up TEXT NOT NULL,
        conexiones_activas INTEGER,
        disk_uso_percent REAL,
        procesos_nombres TEXT,
        procesos_cpu_uso TEXT,
        procesos_memoria TEXT
    )",
    params: [],
)?;
```

Una vez hecho eso se guarda los datos en variables o en vectores dependiendo la métrica ya que para unos se necesita hacer uso de los vectores.

```
let mut usages: Vec<String> = Vec::new();
let mut freqs: Vec<String> = Vec::new();

// Iterar sobre cada núcleo para obtener el uso y la frecuencia.
for (index: usize, cpu: &Cpu) in sys.cpus().iter().enumerate() {
    let uso: f32 = cpu.cpu_usage();
    let frecuencia: u64 = cpu.frequency();

    println!("Núcleo {}: Uso {:.2}%, Frecuencia {} MHz", index, uso, frecuencia);

    usages.push(format!("{:.2}", uso));
    freqs.push(frecuencia.to_string());
}
```

Por ejemplo, ahí se guardaba en un vector el uso del núcleo 1 y del núcleo 2, ya después lo que hice fue separar por comas esto para un manejo más fácil de los datos cuando hiciera las gráficas. El println! solo era para verificar que los datos se estuvieran capturando correctamente.

```
// Separar por comas
let usages_str: String = usages.join(sep: ",");
let freqs_str: String = freqs.join(sep: ",");
```

Hice el mismo procedimiento para los procesos.

```
let mut nombres: Vec<String> = Vec::new();
let mut cpu_uso: Vec<String> = Vec::new();
let mut memoria: Vec<String> = Vec::new();

println!("Top 5 procesos por uso de CPU:");

for (i: usize, process: &&Process) in processes.iter().take(5).enumerate() {
    nombres.push(process.name().to_string_lossy().to_string());
    cpu_uso.push(format!("{:.2}", process.cpu_usage()));
    memoria.push(format!("{:.2}", process.memory() as f64 / (1024.0 * 1024.0)));
    println!(
        "{}. {} (PID: {}) - {:.2}% CPU - {:.2} MB Memoria",
        i + 1,
        process.name().to_string_lossy(),
        process.pid(),
        process.cpu_usage(),
        process.memory() as f64 / (1024.0 * 1024.0)
    );
}
```

Y también para el tráfico.

```
let mut interface: Vec<String> = Vec::new();
let mut data_down: Vec<String> = Vec::new();
let mut data_up: Vec<String> = Vec::new();

for (interface_name: &String, data: &NetworkData) in &networks {
    let download_mb: f64 = data.total_received() as f64 / (1024.0 * 1024.0);
    let upload_mb: f64 = data.total_transmitted() as f64 / (1024.0 * 1024.0);

    interface.push(interface_name.clone());
    data_down.push(format!("{:.2}", download_mb));
    data_up.push(format!("{:.2}", upload_mb));

    println!(
        "{:} ↓ {:.2} MB | ↑ {:.2} MB",
        interface_name,
        download_mb,
        upload_mb
    );
}
```

Para las otras métricas ya era más fácil por que se guardaba un solo dato en la columna como por ejemplo la parte de la memoria.

```
//--2. Metricas de la Memoria--
let mem_fisico: u64 = sys.used_memory() * 100 / sys.total_memory() ;
let mem_swap: u64 = sys.used_swap() * 100 / sys.total_swap() ;

println!("Memoria: \n Memoria fisica {}: \n Memoria swap: {}", mem_fisico, mem_swap);
```

O el disco.

```
// --Porcentaje de uso
let disks: Disks = Disks::new_with_refreshed_list();
let disk_usage: f32 = disks Disks
    .iter() Iter<'_, Disk>
    .map(|disk: &Disk| disk.total_space() - disk.available_space()) impl Iterator<Item = u64>
    .sum::<u64>() as f32
    * 100.0
    / disks Disks
    .iter() Iter<'_, Disk>
    .map(|disk: &Disk| disk.total_space()) impl Iterator<Item = u64>
    .sum::<u64>() as f32;

println!("Disk porcentaje de uso: {:.2}", disk_usage);
```

Una vez capturando los datos lo que hacia era simpleme era pasar los datos a la tabla.

```
conn.execute(
    sql: "INSERT INTO metrics (timestamp, cpu_uso, cpu_frecuencia, mem_fisica, mem_swap, interface_nombre,
                                data_down, data_up, conexiones_activas, disk_uso_percent, procesos_nombres, procesos_cpu_uso,
                                VALUES (?1, ?2, ?3, ?4, ?5, ?6, ?7, ?8, ?9, ?10, ?11, ?12, ?13)",
    params:[
        timestamp,
        usages_str,
        freqs_str,
        mem_fisico,
        mem_swap,
        interface_str,
        data_down_str,
        data_up_str,
        conexiones_activas,
        disk_usage,
        nombres_str,
        cpu_uso_str,
        memoria_str
    ]
)?;
```

Así se veían en el .db:

metrics.db									
Rows: 13									
	id	timestamp	cpu_uso	cpu_frecu...	mem_fisica	mem_swap	interface_nombre		
1	1	2025-04-10 10:32:22	100.00,60.81	2300,2300	84	57	Wi-Fi 2-VirtualBox NDIS Light-Weight Filter-0		
2	2	2025-04-10 10:33:51	100.00,55.31	2300,2300	81	59	Wi-Fi 2,Ethernet 2,Wi-Fi 2-VirtualBox NDIS Lig		
3	3	2025-04-10 10:35:59	65.59,73.08	2300,2300	82	58	Wi-Fi 2-VirtualBox NDIS Light-Weight Filter-0		
4	4	2025-04-10 10:36:42	100.00,82.00	2300,2300	82	58	Wi-Fi 2,Wi-Fi 2-VirtualBox NDIS Light-Weight		
5	5	2025-04-10 10:39:04	100.00,98.33	2300,2300	82	59	Wi-Fi 2-VirtualBox NDIS Light-Weight Filter-0		
6	6	2025-04-10 10:49:52	25.99,26.10	2300,2300	95	20	Wi-Fi 2,Wi-Fi 2-VirtualBox NDIS Light-Weight		
7	7	2025-04-10 10:52:33	19.24,16.15	2300,2300	66	21	Ethernet 2,Wi-Fi 2,Wi-Fi 2-VirtualBox NDIS Lig		
8	8	2025-04-10 11:07:05	64.05,51.35	2300,2300	81	44	Wi-Fi 2-VirtualBox NDIS Light-Weight Filter-0		
9	9	2025-04-10 11:43:36	49.91,73.63	2300,2300	72	38	Wi-Fi 2-VirtualBox NDIS Light-Weight Filter-0		
10	10	2025-04-10 11:46:36	73.06,61.92	2300,2300	71	38	Wi-Fi 2-VirtualBox NDIS Light-Weight Filter-0		

Para que el .exe se ejecutara cada 5 minutos lo que hice fue hacer un archivo.bat y agregarlo a la carpeta de inicio de Windows para que se ejecutara al iniciar el sistema.

```
@echo off
:loop
    "C:\Users\Julio\Desktop\monitor_sistema_per\target\debug
\monitor_sistema_per.exe"
    timeout /t 300|
    goto loop
```

Lo que seguía era pasarlo a .csv una vez con esto lo pasaba a pyhon.

```
df = pd.read_csv("metrics.csv", sep=";")
```

Primero se extraen los datos para ya después con eso poder graficar.

```
df["tiempo"] = pd.to_datetime(df["timestamp"])
df["cpu_uso_0"], df["cpu_uso_1"] = zip(*df["cpu_uso"].str.split(",").map(lambda x: map(float, x)))
df["cpu_freq_0"], df["cpu_freq_1"] = zip(*df["cpu_frecuencia"].str.split(",").map(lambda x: map(float, x)))

df["mem_fisica_porcentaje"] = df["mem_fisica"].astype(float)
df["mem_swap_porcentaje"] = df["mem_swap"].astype(float)
df["disco_uso_porcentaje"] = df["disk_uso_percent"].astype(float)
df["conexiones_activas"] = df["conexiones_activas"].astype(int)
```

Para los procesos y el trafico es un poco diferente ya que hay mucho datos en una sola columna por lo que es mejor separar los datos de esta manera.

```
trafico = []

for idx, row in df.iterrows():
    timestamp = row["timestamp"]

    nombres = row["interface_nombre"].split(",")
    downs = list(map(float, row["data_down"].split(",")))
    ups = list(map(float, row["data_up"].split(",")))

    for nombre, down, up in zip(nombres, downs, ups):
        trafico.append({
            "timestamp": timestamp,
            "interfaz": nombre,
            "data_down": down,
            "data_up": up
        })

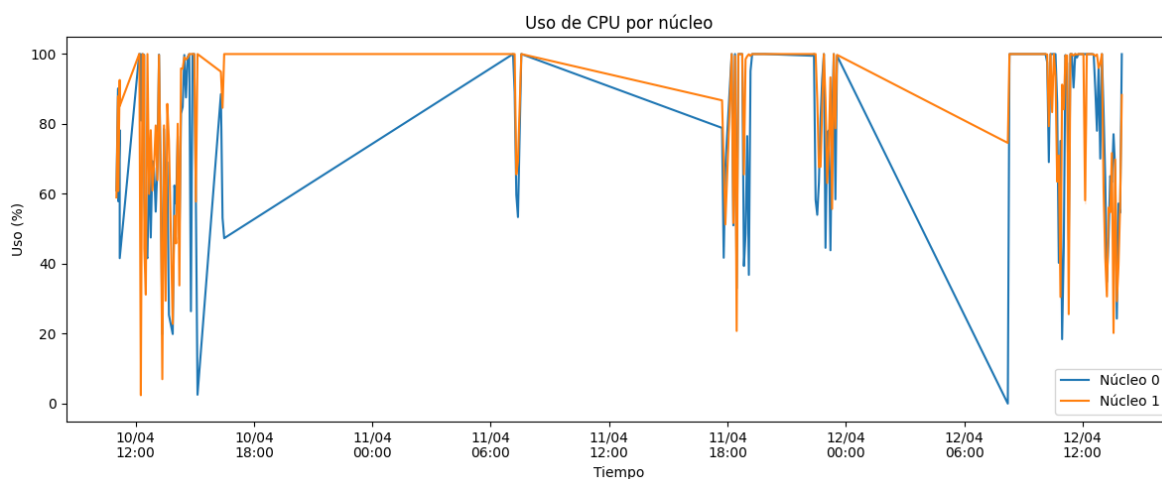
trafico_df = pd.DataFrame(trafico)
```

Con esto lo que se logra es que se vea de la siguiente manera:

	data_down	data_up
interfaz		
Ethernet 2	0.000000	0.00
Wi-Fi 2	35.638462	4.26
Wi-Fi 2-VirtualBox NDIS Light-Weight Filter-0000	35.638462	4.26

Una vez pasando los datos a nuestra data frame podemos comenzar con la graficas.

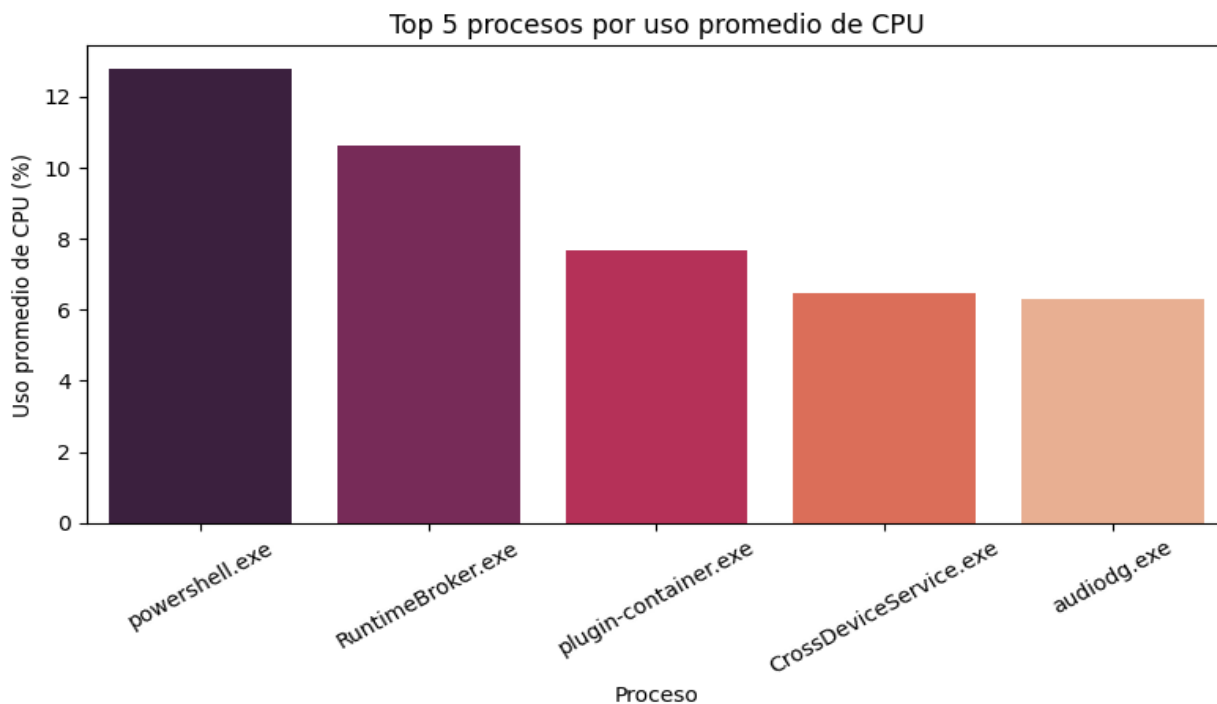
Aquí se puede ver el uso de la cpu en cada uno de los núcleos.



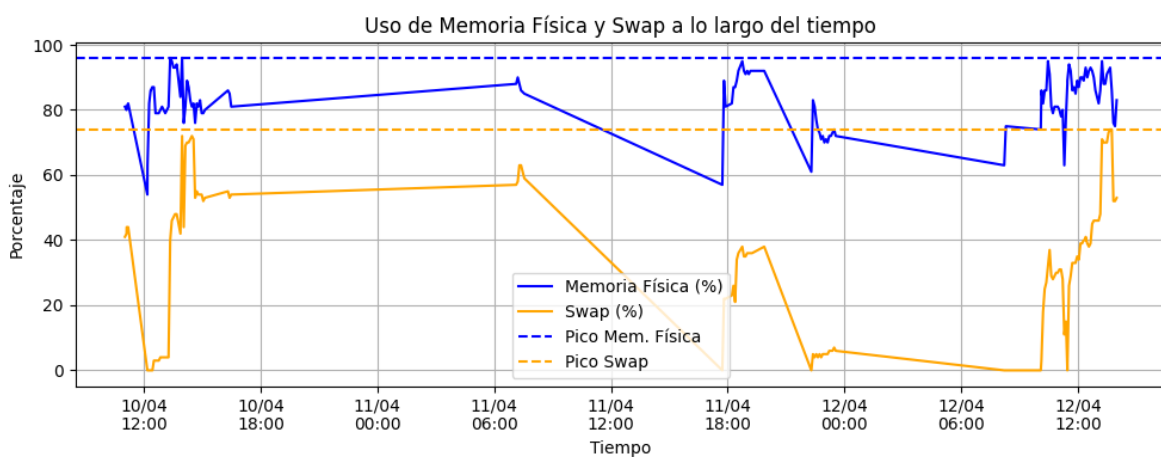
Y saber cual fue su pico y promedio de uso.

```
Promedio CPU núcleo 0: 77.47608108108108  
Pico CPU núcleo 0: 100.0  
Promedio CPU núcleo 1: 81.41033783783783  
Pico CPU núcleo 1: 100.0
```

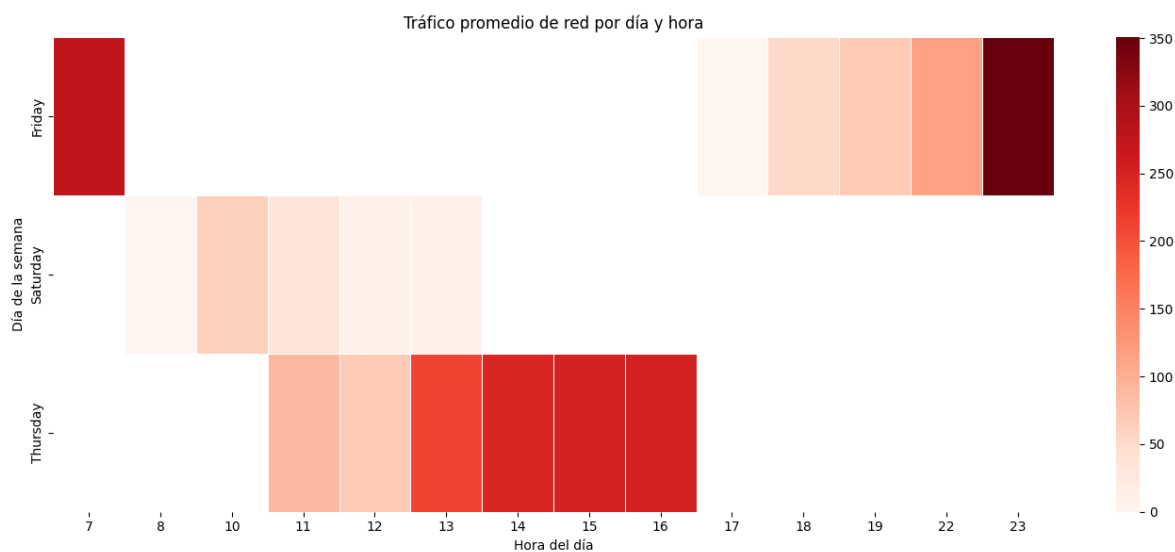
Saber cuáles cual fue el top 5 de procesos por el promedio de uso de cpu.



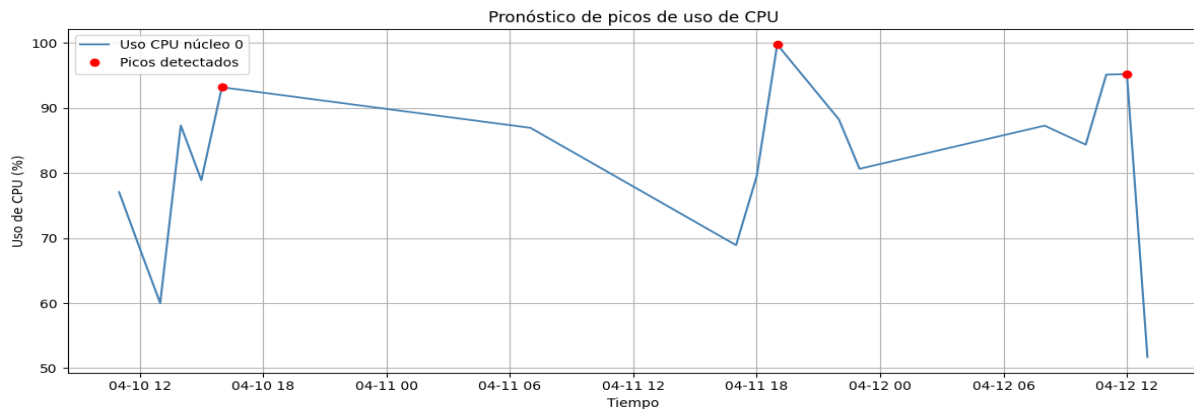
Mostrar el uso de la memoria física y swap a través del tiempo.



Además de mostrar con un mapa de calor el tráfico de red promedio por día y hora, donde se observa que el horario crítico durante esto 2/3 días fue el viernes en la noche.



Por último, una gráfica y una proyección sobre los picos de la cpu.



Se espera un pico de CPU cada 22.0 horas.

Se detecta un pico de uso de CPU cada **22 horas**, lo que indica un patrón periódico. Esto permite anticipar momentos de alta carga y optimizar el rendimiento del sistema.

Conclusión

Este trabajo me costó mucho en la parte de rust conseguir los datos, ya que me causaban muchos problemas las versiones de las librerías, porque una vez teniendo los datos se me hizo mas familiar graficar gracias a que en la materia de aprendizaje maquina nos están enseñando las librerías que use como pandas, seaborn, matplotlib.pyplot y numpy.

Otra cosa que me costo resolver fue que el programa se ejecutara cada cierto tiempo, en un inicio lo había hecho creando una tarea, el problema es que no veía donde se generaba el archivo.db lo que tuve que hacer fue investigar cómo se hacían los programas.bat de esa manera fue mas sencillo ya que lo guarde en el escritorio y ahí pude ver como estaba el archivo.db creado con varias inyecciones.

En términos generales me gusto, aunque lo pude haber hecho mas completo, ya que me faltaron algunas cosas de capturar.

Referencias.

Bos, R. (2023, 8 mayo). *Use after free vulnerability in C++ | Tutorial & Examples*. Snyk Learn. <https://learn.snyk.io/lesson/use-after-free/>

Crates.io: Rust package Registry. (s. f.-b). crates.io: Rust Package Registry. <https://crates.io/crates/rusqlite>

Crates.io: Rust package Registry. (s. f.-c). crates.io: Rust Package Registry. <https://crates.io/crates/csv>